

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC: 30724
	Apellidos y nombres del estudiante: • Díaz Stiven • Leiton Jose • Manosalvas Gabriel

ANTECEDENTES

AliGest nació como respuesta a una necesidad concreta del Condominio La Primavera, donde la administración de expensas se realizaba de forma completamente manual, apoyándose en hojas de cálculo Excel para registrar pagos, calcular recargos por mora del 12% y llevar el seguimiento de los 60 copropietarios del conjunto, lo cual generaba errores frecuentes, falta de trazabilidad y una carga operativa significativa para el Administrador, al mismo tiempo que los copropietarios no contaban con un canal transparente y confiable para verificar el estado de sus obligaciones, en ese contexto el grupo 6 decidió que va a construir una plataforma web bajo la metodología XP que centralizará todas estas operaciones en un solo lugar, abarcando desde la autenticación de usuarios hasta el envío automático de notificaciones por WhatsApp vinculadas a la aprobación, rechazo o sanción por mora de cada pago registrado.

OBJETIVO

Justificar la implementación del patrón de diseño Command en el sistema AliGest, demostrando cómo este patrón permite encapsular las operaciones críticas en objetos independientes, facilitando la trazabilidad de acciones, el desacoplamiento entre componentes y la mantenibilidad del código.

DESARROLLO

Pienso que la mejor manera de entender por qué el patrón Command es importante para AliGest es entendiendo que hace este patrón, Command encapsula una solicitud o acción como un objeto, permitiendo que el emisor de esa acción no necesite saber nada sobre quién la ejecutará ni cómo lo hará, lo cual genera un nivel de desacoplamiento muy valioso en sistemas donde las operaciones son diversas y deben ser trazables.

Considerando los requisitos funcionales del sistema, especialmente los relacionados con RF-03 Implementar Pagos y RF-04 Enviar Notificaciones, se puede observar que AliGest tiene un flujo de operaciones encadenadas y dependientes entre sí, por ejemplo cuando un copropietario registra un pago se desencadenan múltiples acciones como la validación del comprobante, la detección automática de mora, el cálculo del recargo del 12%, la generación del comprobante digital y finalmente el envío de la notificación por WhatsApp, y pienso que aplicar Command aquí permite encapsular cada una de esas acciones como objetos independientes que el sistema puede ejecutar, encolar, registrar en logs de auditoría e incluso revertir si fuera necesario.

Otro punto que considero fundamental para justificar este patrón es la restricción de seguridad RNF-01 y RNF-02 del SRS, que exigen seguridad en la autenticación e integridad en los datos y transacciones, ya que al encapsular cada operación como un Command el sistema puede registrar automáticamente quién ejecutó cada acción, en qué momento y con qué parámetros, lo que genera un log de auditoría natural.

Pienso también que el contexto de desarrollo en XP favorece enormemente la adopción de Command, ya que esta metodología exige iteraciones cortas, pruebas continuas y refactorización permanente, y el hecho de que cada operación sea un objeto independiente facilita enormemente escribir pruebas unitarias para cada comando de forma aislada.

Considerando la evolución previsible del sistema descrita en el SRS, donde se plantea la integración futura con pasarelas de pago, APIs bancarias y una arquitectura multicondominio SaaS, se puede ver que Command también prepara el terreno para esas ampliaciones porque al tener las operaciones encapsuladas en objetos es mucho más sencillo agregar nuevos comandos sin afectar los existentes, respetando el principio abierto-cerrado que es clave para proyectos que aspiran a escalar.

CONCLUSIONES

- El patrón Command responde de manera directa a los desafíos técnicos y operativos que AliGest enfrenta, pues encapsula cada operación del sistema como un objeto independiente que puede ser ejecutado, registrado y auditado, lo que se traduce en mayor trazabilidad de las acciones del Administrador y los copropietarios, mayor facilidad para escribir pruebas unitarias en el marco de XP y una arquitectura más limpia y mantenible que soportará las futuras integraciones planificadas para el sistema.
- Se considera que sin un patrón como Command el sistema estaría expuesto a un alto acoplamiento entre las capas de la aplicación, lo que dificultará tanto el mantenimiento a corto plazo como la escalabilidad que el proyecto necesita a mediano y largo plazo según su hoja de ruta tecnológica.

RECOMENDACIONES

- Se recomienda aplicar el patrón Command principalmente en los módulos de pagos y notificaciones, que son los más complejos y críticos del sistema, asegurándose de que cada comando cuente con su respectiva clase, interfaz y prueba unitaria desde la primera iteración del proyecto para no acumular deuda técnica.
- Documentar en el repositorio del proyecto un catálogo de los comandos implementados con su descripción, parámetros y efecto esperado, de modo que el equipo pueda mantener coherencia en las futuras iteraciones y que el Administrador del condominio pueda comprender qué operaciones están disponibles en el sistema sin necesidad de revisar el código fuente directamente.

Sangolquí, a 28 de Mayo de 2026